

Tech Stack

Java 17

Spring Boot

Spring Security

Spring Data JPA

PostgreSQL

Redis

Docker

Maven

Step 1: Create Spring Boot Project

Dependencies to Add

Add these dependencies while creating the project from:

Spring Initializr

Required Dependencies

Spring Web

Spring Data JPA

Spring Security

PostgreSQL Driver

Validation

Lombok

Spring Boot DevTools

Redis

Docker Compose Support

Step 2: Project Structure

smart-social-backend/

```
|
|├── src/
| |├── main/
| | |├── java/com/socialmedia/
| | | |
| | |├── controller/
| | |├── service/
| | |├── repository/
| | |├── entity/
| | |├── dto/
| | |├── security/
| | |├── config/
| | |├── exception/
| | |├── notification/
| | |├── moderation/
| | |├── virality/
| | |└── util/
| | |
| | |└── SmartSocialBackendApplication.java
| |
| |└── resources/
| | |├── application.properties
| | |└── data.sql
|
|├── docker-compose.yml
|├── Dockerfile
|├── pom.xml
|└── README.md
```

Step 3: pom.xml Dependencies

```
<dependencies>
```

```
    <!-- Spring Web -->
```

```
    <dependency>
```

```
        <groupId>org.springframework.boot</groupId>
```

```
        <artifactId>spring-boot-starter-web</artifactId>
```

```
    </dependency>
```

```
    <!-- Spring Security -->
```

```
    <dependency>
```

```
        <groupId>org.springframework.boot</groupId>
```

```
        <artifactId>spring-boot-starter-security</artifactId>
```

```
    </dependency>
```

```
    <!-- Spring Data JPA -->
```

```
    <dependency>
```

```
        <groupId>org.springframework.boot</groupId>
```

```
        <artifactId>spring-boot-starter-data-jpa</artifactId>
```

```
    </dependency>
```

```
    <!-- PostgreSQL -->
```

```
    <dependency>
```

```
        <groupId>org.postgresql</groupId>
```

```
        <artifactId>postgresql</artifactId>
```

```
        <scope>runtime</scope>
```

```
    </dependency>
```

```
    <!-- Redis -->
```

```
    <dependency>
```

```
        <groupId>org.springframework.boot</groupId>
```

```
        <artifactId>spring-boot-starter-data-redis</artifactId>
```

```
    </dependency>
```

```
<!-- Validation -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

```
<!-- Lombok -->
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <optional>true</optional>
</dependency>
```

```
<!-- JWT -->
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.11.5</version>
</dependency>
```

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.11.5</version>
  <scope>runtime</scope>
</dependency>
```

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.11.5</version>
  <scope>runtime</scope>
</dependency>
```

</dependencies>

Step 4: application.properties

Application

spring.application.name=smart-social-backend

PostgreSQL Configuration

spring.datasource.url=jdbc:postgresql://localhost:5432/socialmedia

spring.datasource.username=postgres

spring.datasource.password=postgres

JPA Configuration

spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true

spring.jpa.properties.hibernate.format_sql=true

Redis Configuration

spring.data.redis.host=localhost

spring.data.redis.port=6379

Server

server.port=8080

JWT

jwt.secret=smartSocialMediaSecretKey

jwt.expiration=86400000

Step 5: docker-compose.yml

version: '3.8'

services:

postgres:

image: postgres:16

```
container_name: vironix-postgres
environment:
  POSTGRES_DB: vironix_db
  POSTGRES_USER: postgres
  POSTGRES_PASSWORD: postgres
ports:
  - "5433:5432"
volumes:
  - postgres_data:/var/lib/postgresql/data
redis:
  image: redis:7
  container_name: vironix-redis
  ports:
    - "6379:6379"
  volumes:
    postgres_data:
```

Step 6: Build the Project

Open terminal in project root.

Run:

```
mvn clean install
```

This creates:

```
target/smart-social-backend.jar
```

Step 7: Run Using Docker

Start Containers

```
docker-compose up --build
```

Step 8: Verify Running Containers

```
docker ps
```

You should see:

PostgreSQL container

Redis container

Backend container

Step 9: Test Backend

Open browser:

`http://localhost:8080`

Or test APIs using:

Postman

Swagger

Step 10: Useful Docker Commands

Stop Containers

`docker-compose down`

View Logs

`docker logs smart-social-backend`

Restart Containers

`docker-compose restart`